CO1 ASSIGNMENT

NAME: Tharani RB
REG NO:1911260138
COURSE CODE:UBA0822
COURSE NAME:PYTHON PROGRAMMING
SLOT:B
GITHUB: **GitHub - TharaniRB15/CSA0822-1911260138-**

TASK 1-Milk Payment Problem Analysis

Aim:

This program calculates the milk payment for farmers. It gets the farmer ID, milk quantity, fat percentage, SNF percentage, and shift as inputs. It checks whether the inputs are valid, calculates the payment using the rate per litre, and adds the payments to find the total amount

Program:

File Edit Format Run Options Window Help

```
minfat=2
maxfat=7
totaldays=10
rate=35
totalamount=0
n=int(input('no.of farmers'))
for i in range(n):
    print('farmer',i+1)
    fid=int(input('enter id'))
    litre=float(input('enter litre'))
    fat=float(input('enter fat'))
    snf=float(input('enter snf'))
    shift=input('morning/evening')
    if litre<0:
        print('invalid')
    elif fat < minfat or fat > maxfat:
        print('invalid')
    elif shift not in('morning','evening'):
        print('invalid')
    else:
        amount=litre*rate
        totalamount=totalamount+amount
        print('id',fid)
        print('rate',rate)
        print('amount',amount)
print('total',totalamount)
```

output:

```
========== RESTART: C:/Users/thara/OneDrive/Attachments/PYTHON/ba.py ==========
no.of farmers3
farmer 1
enter id101
enter litre2
enter fat3
enter snf4
morning/eveningmorning
id 101
rate 35
amount 70.0
farmer 2
enter id102
enter litre3
enter fat4
enter snf5
morning/eveningmorning
id 102
rate 35
amount 105.0
farmer 3
enter id103
enter litre5
enter fat6
enter snf4
morning/eveningevening
id 103
rate 35
amount 175.0
total 350.0
```

TASK-2 Python Data Types and Precision

Aim:

This program explores Python data types, division operators, and floating-point errors. It calculates the total of 1,000 payments and three adjustments using both float and Decimal. Float may produce small precision errors, whereas Decimal gives the exact money total. Therefore, Decimal is better for accurate financial calculations.

Program

ba.py - C:/Users/thara/OneDrive/Attachments/python practise/ba.py (3.14.8)

File Edit Format Run Options Window Help

```
from decimal import Decimal
payments = ["33.40"] * 1000 + ["0.10"] * 3
def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total
def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total
print("float:", total_float(payments))
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3)
print(Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
print("Decimal from float:", Decimal(float("0.1")))
```

output

DLE Shell 3.14.8

Edit Shell Debug Options Window Help

```
Python 3.14.8 (tags/v3.14.8:8e6e75d, Sep 30 2026, 18:19:33) [MSC v.1944 64 bit (AMD64)] on win3
Enter "help" below or click "Help" above for more information.

========== RESTART: C:/Users/thara/OneDrive/At
float: 33400.30000000061
Decimal: 33400.30
False
True
Decimal from float: 0.1000000000000000055511151231257827021181583404541015625
```

TASK-3 Variables and Constants

Aim:

This program uses meaningful variable names and uppercase named constants to manage milk payment rates and limits. Python supports dynamic typing and multiple assignment. Using a dictionary to store rates makes future rate revisions easier because the values can be changed in one place.

Program:

File Edit Format Run Options Window Help

```
BASE_RATE = 30.0
FAT_RATE = 0.40
SNF_RATE = 0.30
MIN_FAT = 2.0
MAX_FAT = 7.0
TOTAL_DAYS = 10
RATE_CARD = {
    "base": BASE_RATE,
    "fat": FAT_RATE,
    "snf": SNF_RATE}
farmer_id = int(input("Enter farmer ID: "))
litres = float(input("Enter litres: "))
fat = float(input("Enter fat percentage: "))
snf = float(input("Enter SNF percentage: "))
if litres < 0:
    print("Invalid litres")
elif fat < MIN_FAT or fat > MAX_FAT:
    print("Invalid fat percentage")
else:
    rate = (RATE_CARD["base"]
            + fat * RATE_CARD["fat"]
            + snf * RATE_CARD["snf"])
    amount = litres * rate
    print("Farmer ID:", farmer_id)
    print("Rate per litre:", round(rate, 2))
    print("Payment amount:", round(amount, 2))
    print("Ten-day total (if daily amount is same):",
          round(amount * TOTAL_DAYS, 2))
value = 10
print(type(value))
value = "Morning"
print(type(value))
a, b = 10, 20
a, b = b, a
print("After swapping:", a, b)
```

output

```
Python 3.14.8 (tags/v3.14.8:8e6e75d, Sep 30 2026, 18:19:3
Enter "help" below or click "Help" above for more informa

========== RESTART: C:/Users/thara/OneDrive/Attachments/pythc
Enter farmer ID: 101
Enter litres: 2
Enter fat percentage: 3
Enter SNF percentage: 5
Farmer ID: 101
Rate per litre: 32.7
Payment amount: 65.4
Ten-day total (if daily amount is same): 654.0
<class 'int'>
<class 'str'>
After swapping: 20 10
```

TASK-4 Mathematical Expressions and Operators

Aim:

This program calculates fat and SNF steps using round(), determines the rate per litre, and calculates the payment amount. It demonstrates operator precedence, floating-point precision errors, comparison chaining, and the differences between round(), int(), and math.floor().

Program:

File Edit Format Run Options Window Help

```
import math
BASE_RATE = 30.0
FAT_STEP_RATE = 0.40
SNF_STEP_RATE = 0.30
FAT_BASE = 3.5
SNF_BASE = 8.5
STEP_SIZE = 0.1
fat = float(input("Enter fat percentage: "))
snf = float(input("Enter SNF percentage: "))
litres = float(input("Enter litres: "))
fat_steps = round((fat - FAT_BASE) / STEP_SIZE)
snf_steps = round((snf - SNF_BASE) / STEP_SIZE)
rate = (BASE_RATE
        + fat_steps * FAT_STEP_RATE
        + snf_steps * SNF_STEP_RATE)
amount = litres * rate
print("Fat steps:", fat_steps)
print("SNF steps:", snf_steps)
print("Rate per litre:", round(rate, 2))
print("Total amount:", round(amount, 2))
fat_value = 4.2
print("Round:", round((fat_value - FAT_BASE) / STEP_SIZE))
print("Int:", int((fat_value - FAT_BASE) / STEP_SIZE))
print("Floor:", math.floor((fat_value - FAT_BASE) / STEP_SIZE))
fat_value = 3.1
print("Round:", round((fat_value - FAT_BASE) / STEP_SIZE))
print("Int:", int((fat_value - FAT_BASE) / STEP_SIZE))
print("Floor:", math.floor((fat_value - FAT_BASE) / STEP_SIZE))
print(0.1 + 0.2 == 0.3)
print(int(-3.7))
print(math.floor(-3.7))
```

OUTPUT:

```
Python 3.14.8 (tags/v3.14.8:8e6e75d, Sep 30
Enter "help" below or click "Help" above fo

======= RESTART: C:/Users/thara/OneDrive/Att
Enter fat percentage: 4
Enter SNF percentage: 3
Enter litres: 2
Fat steps: 5
SNF steps: -55
Rate per litre: 15.5
Total amount: 31.0
Round: 7
Int: 7
Floor: 7
Round: -4
Int: -3
Floor: -4
False
-3
-4
```

TASK-5 Input and Type Conversion

Aim:

This program reads the farmer's details using input(), converts them into appropriate data types, validates the values, and calculates the payment. The try-except block handles invalid numerical inputs, while f-strings display the results neatly with two decimal places.

Program:

ba.py - C:/Users/thara/OneDrive/Attachments/python practise/ba.py (3.14.8)

File Edit Format Run Options Window Help

```
try:
    farmer_id = int(input("Enter farmer ID: "))
    litres = float(input("Enter litres: ").replace(",", "."))
    fat = float(input("Enter fat percentage: ").replace(",", "."))
    snf = float(input("Enter SNF percentage: ").replace(",", "."))
    rate = float(input("Enter rate per litre: ").replace(",", "."))
    if litres < 0 or not (2.0 <= fat <= 7.0) or snf < 0 or rate < 0:
        print("Invalid input")
    else:
        amount = litres * rate
        print(f"Farmer ID: {farmer_id}")
        print(f"Litres: {litres:8.2f}")
        print(f"Fat: {fat:8.2f}%")
        print(f"SNF: {snf:8.2f}%")
        print(f"Amount: Rs.{amount:8.2f}")
except ValueError:
    print("Invalid input! Enter numbers correctly.")
```

output:

```
========== RESTART: C:/Users/thara/OneDr
Enter farmer ID: 101
Enter litres: 2
Enter fat percentage: 3
Enter SNF percentage: 4
Enter rate per litre: 5
Farmer ID: 101
Litres:      2.00
Fat:      3.00%
SNF:      4.00%
Amount: Rs.    10.00
>>>
```

TASK-6 Functions and Code Reusability

Aim:

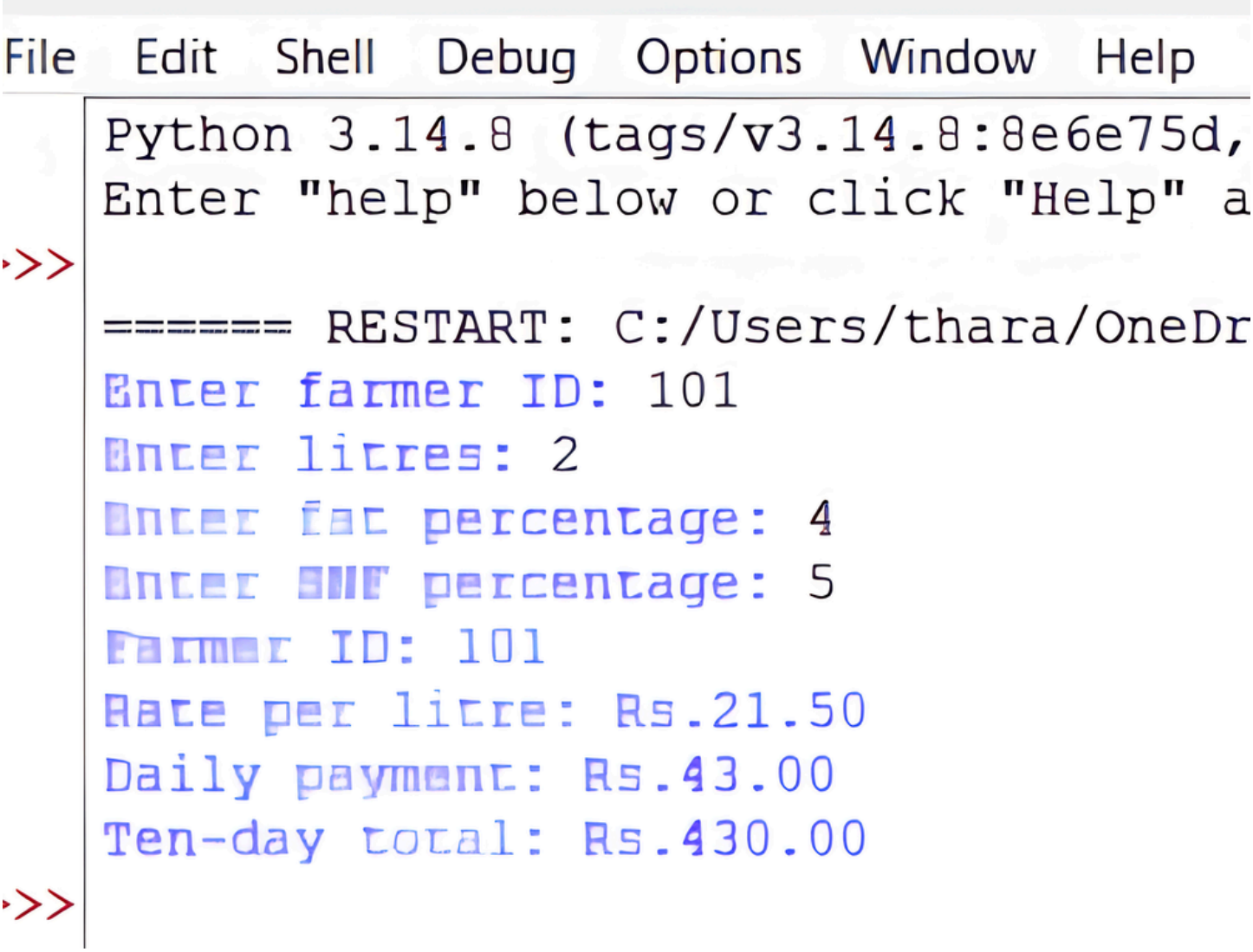
This program divides the milk payment calculation into three functions: rate_per_litre(), payout(), and ten_day_total(). Each function accepts parameters and returns a value. Global constants store fixed rates, local variables store temporary values, and docstrings explain each function. Using functions improves code reuse, readability, and maintenance. The time complexity is \(O(1)\) for each calculation and \(O(n)\) when processing \(n\) farmer entries.

Program:

File Edit Format Run Options Window Help

```
BASE_RATE = 30.0
FAT_RATE = 0.40
SNF_RATE = 0.30
FAT_BASE = 3.5
SNF_BASE = 8.5
STEP_SIZE = 0.1
TOTAL_DAYS = 10
def rate_per_litre(fat, snf):
    """Calculate milk rate per litre using fat and SNF steps."""
    fat_steps = round((fat - FAT_BASE) / STEP_SIZE)
    snf_steps = round((snf - SNF_BASE) / STEP_SIZE)
    rate = (BASE_RATE
            + fat_steps * FAT_RATE
            + snf_steps * SNF_RATE)
    return rate
def payout(litres, rate):
    """Calculate payment for the supplied milk."""
    return litres * rate
def ten_day_total(daily_amount):
    """Calculate total payment for ten days at the same daily amount."""
    return daily_amount * TOTAL_DAYS
farmer_id = int(input("Enter farmer ID: "))
litres = float(input("Enter litres: "))
fat = float(input("Enter fat percentage: "))
snf = float(input("Enter SNF percentage: "))
if litres < 0 or not (2.0 <= fat <= 7.0) or snf < 0:
    print("Invalid input")
else:
    rate = rate_per_litre(fat, snf)
    amount = payout(litres, rate)
    total = ten_day_total(amount)
    print("Farmer ID:", farmer_id)
    print(f"Rate per litre: Rs.{rate:.2f}")
    print(f"Daily payment: Rs.{amount:.2f}")
    print(f"Ten-day total: Rs.{total:.2f}")
```

output



TASK-7 Python Modules and Imports

Aim:

A Python module stores reusable constants and functions in a separate file. The main script imports the module and calls its functions. The if __name__ == "__main__": condition prevents the demonstration table from printing during import, while allowing it to print when the module is run directly. This improves code reuse and makes rate revisions easier.

Program

```
File  Edit  Format  Run  Options  Window  Help
"""Rate card of the village milk co-operative."""
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
def rate_per_litre(fat, snf):
    """Calculate rate per litre."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    rate = (BASE_RATE
            + fat_steps * FAT_STEP_VALUE
            + snf_steps * SNF_STEP_VALUE)
    return round(rate, 2)
def payout(litres, fat, snf):
    """Calculate farmer payment."""
    return round(litres * rate_per_litre(fat, snf), 2)
if __name__ == "__main__":
    for litres, fat, snf in [
        (10, 3.5, 8.5),
        (8.5, 4.2, 8.7),
        (12, 3.1, 8.3)
    ]:
        rate = rate_per_litre(fat, snf)
        pay = payout(litres, fat, snf)
        print(f"{litres:g} L fat {fat} snf {snf} rate {rate:.2f} pay {pay:.2f}")
```

output

```
====== RESTART: C:/Users/thara/OneDrive/Att
10 L fat 3.5 snf 8.5 rate 30.00 pay 300.00
8.5 L fat 4.2 snf 8.7 rate 33.40 pay 283.90
12 L fat 3.1 snf 8.3 rate 27.80 pay 333.60
```

TASK-8 Algorithm Design and Planning

Aim:

This algorithm accepts the farmer's milk details for ten days, validates each entry, calculates the rate and payment, and maintains a running total. It uses $O(n)$ time and $O(1)$ extra space for n entries.

Program:

File Edit Format Run Options Window Help

```
BASE_RATE = 30.0
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
TOTAL_DAYS = 10
def rate_per_litre(fat, snf):
    """Calculate the rate per litre."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = (BASE_RATE
            + fat_steps * FAT_STEP_VALUE
            + snf_steps * SNF_STEP_VALUE)
    return round(rate, 2)
farmer_id = input("Enter farmer ID: ")
ten_day_total = 0.0
for day in range(1, TOTAL_DAYS + 1):
    print("Day", day)
    litres = float(input("Enter litres: "))
    fat = float(input("Enter fat %: "))
    snf = float(input("Enter SNF %: "))
    if litres < 0 or not (2 <= fat <= 7) or snf < 0:
        print("Invalid entry")
        continue
    rate = rate_per_litre(fat, snf)
    amount = round(litres * rate, 2)
    ten_day_total = round(ten_day_total + amount, 2)
    print(f"Rate: Rs.{rate:.2f}")
    print(f"Amount: Rs.{amount:.2f}")
print("Farmer ID:", farmer_id)
print(f"Ten-day total: Rs.{ten_day_total:.2f}")
```

output

```
====== RESTART: C:/Users/thara/OneDrive/I
Enter farmer ID: 101
Day 1
Enter litres: 3
Enter fat %: 2
Enter SNF %: 4
Rate: Rs.10.50
Amount: Rs.31.50
Day 2
Enter litres:
```

TASK-9 Implementation and Testing

Aim:

This program calculates milk payment using fat and SNF values. It tests normal, boundary, and invalid entries using assert statements. The five test payments total ₹1301.50. Automated tests can be rerun quickly after every code change, whereas manual testing requires entering each input again. Processing \(n\) entries takes \(O(n)\) time, and storing the test cases takes \(O(n)\) space.

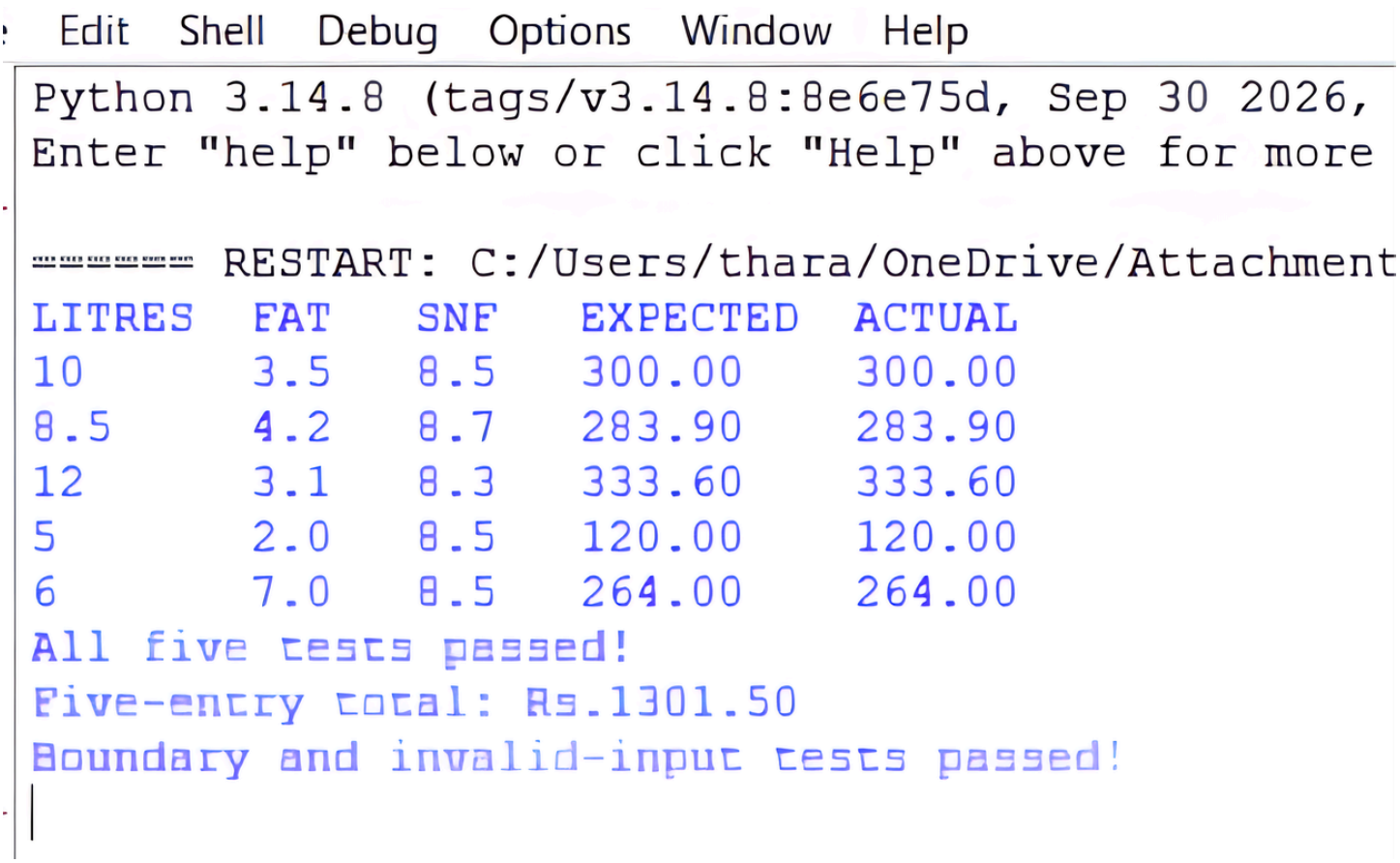
Program:

```
BASE_RATE = 30.0
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
def rate_per_litre(fat, snf):
    """Calculate milk rate per litre."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    return round(
        BASE_RATE
        + fat_steps * FAT_STEP_VALUE
        + snf_steps * SNF_STEP_VALUE, 2
    )
def payout(litres, fat, snf):
    """Calculate payment after validating the inputs."""
    if litres < 0:
        raise ValueError("Litres cannot be negative")
    if not 2.0 <= fat <= 7.0:
        raise ValueError("Fat must be between 2% and 7%
    if snf < 0:
        raise ValueError("SNF cannot be negative")

    return round(litres * rate_per_litre(fat, snf), 2)
tests = [
    (10, 3.5, 8.5, 300.00),
    (8.5, 4.2, 8.7, 283.90),
    (12, 3.1, 8.3, 333.60),
    (5, 2.0, 8.5, 120.00),
    (6, 7.0, 8.5, 264.00)
]
total = 0.0
print("LITRES  FAT   SNF   EXPECTED  ACTUAL")
for litres, fat, snf, expected in tests:
    actual = payout(litres, fat, snf)
    assert actual == expected
    total += actual
    print(f"{litres:<7} {fat:<5} {snf:<5} "
          f"{expected:<9.2f} {actual:.2f}")
print("All five tests passed!")
print(f"Five-entry total: Rs.{total:.2f}")
assert rate_per_litre(3.5, 8.5) == 30.00
```

```
assert payout(1, 2.0, 8.5) == 24.00
assert payout(1, 7.0, 8.5) == 44.00
for bad_entry in [
    (-1, 3.5, 8.5),
```

Output:



TASK 10-Comparing Expected and Actual Results

Aim:

This task compares float and Decimal calculations for 300, 6,000, and 150,000 milk payments. It checks the difference in totals, execution time, and whether rounding each payment or only the final total gives the correct bank amount. It also compares recalculating rates with storing them in a dictionary.

Program:

File Edit Format Run Options Window Help

```
from decimal import Decimal, ROUND_HALF_UP
import time
RATE = 40.50
FAT_RATE = 2.0
SNF_RATE = 1.0
def float_payment(n):
    total = 0.0
    for _ in range(n):
        total += RATE * FAT_RATE + RATE * SNF_RATE
    return total
def decimal_payment(n, round_each=False):
    total = Decimal("0")
    payment = Decimal("40.50") * Decimal("2") + Decimal("40.50") * Decimal("1")
    for _ in range(n):
        if round_each:
            payment_used = payment.quantize(
                Decimal("0.01"), rounding=ROUND_HALF_UP
            )
        else:
            payment_used = payment
        total += payment_used
    return total.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
for n in [300, 6000, 150000]:
    start = time.perf_counter()
    f_total = float_payment(n)
    float_time = time.perf_counter() - start
    start = time.perf_counter()
    d_total = decimal_payment(n)
    decimal_time = time.perf_counter() - start
    print(f"\nPayments: {n}")
    print(f"Float total: {f_total:.2f}")
    print(f"Decimal total: {d_total}")
    print(f"Difference: {abs(Decimal(str(f_total)) - d_total):.6f}")
    print(f"Float time: {float_time:.6f} seconds")
    print(f"Decimal time: {decimal_time:.6f} seconds")
    print(f"Rounded each payment: {decimal_payment(n, True)}")
```

output:

```
====== RESTART: C:/Users/thara/OneDrive/Attachments/p

Payments: 300
Float total: 36450.00
Decimal total: 36450.00
Difference: 0.000000
Float time: 0.000065 seconds
Decimal time: 0.000180 seconds
Rounded each payment: 36450.00

Payments: 6000
Float total: 729000.00
Decimal total: 729000.00
Difference: 0.000000
Float time: 0.001245 seconds
Decimal time: 0.001367 seconds
Rounded each payment: 729000.00

Payments: 150000
Float total: 18225000.00
Decimal total: 18225000.00
Difference: 0.000000
Float time: 0.023864 seconds
Decimal time: 0.035810 seconds
Rounded each payment: 18225000.00
```

TASK-11 Final Design Recommendation and Evaluation

Aim:

This task recommends the best milk payment calculator design for the co-operative. It explains why Decimal and proper rounding are suitable, how milk rates can be updated using a JSON file, the limitations of the program, and whether it can handle district-union-level payments.

Program:

ba.py - C:/Users/thara/OneDrive/Attachments/python practise/ba.py (

File Edit Format Run Options Window Help

```
from decimal import Decimal, ROUND_HALF_UP
import time

def calculate_payment(fat, snf, quantity):
    fat_rate = Decimal("2.00")
    snf_rate = Decimal("1.00")

    payment = (
        Decimal(str(fat)) * fat_rate
        + Decimal(str(snf)) * snf_rate
    ) * Decimal(str(quantity))

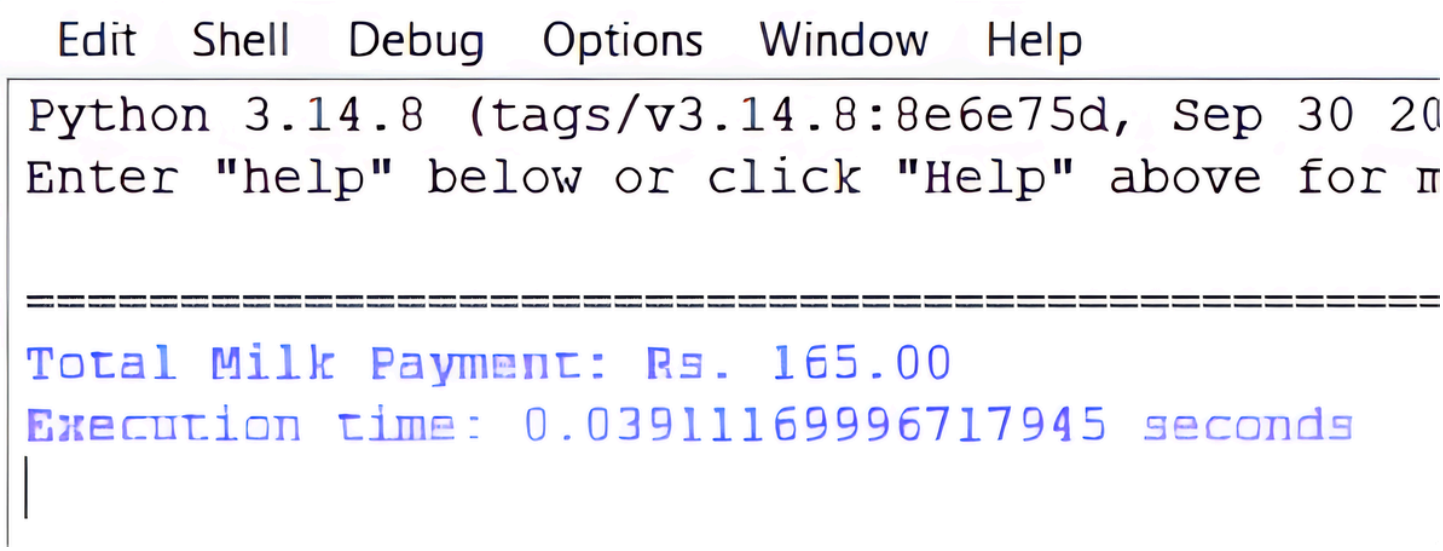
    return payment.quantize(
        Decimal("0.01"),
        rounding=ROUND_HALF_UP
    )

start = time.perf_counter()

amount = calculate_payment(4.0, 8.5, 10)

print("Total Milk Payment: Rs.", amount)
print("Execution time:",
      time.perf_counter() - start, "seconds")
```

Output:



TASK-12 Re-evaluation and Adaptation of the Design

Aim:

The milk payment calculator must be updated to support two-decimal FAT and SNF readings, seasonal rates from April to June, and a bonus of ₹0.50 per litre when a farmer supplies more than 500 litres in 10 days. Decimal is used for accurate calculations and rounding.

Program:

File Edit Format Run Options Window Help

```
from decimal import Decimal, ROUND_HALF_UP
from datetime import date
def calculate_payment(fat, snf, litres, total_10_days, month):
    base_rate = Decimal("32.00")
    fat_step_rate = Decimal("0.45")
    bonus_rate = Decimal("0.50")
    fat = Decimal(str(fat)).quantize(Decimal("0.01"))
    snf = Decimal(str(snf)).quantize(Decimal("0.01"))
    litres = Decimal(str(litres))
    if month in [4, 5, 6]:
        rate = base_rate + (fat * fat_step_rate)
    else:
        rate = base_rate + (fat * fat_step_rate)
    if Decimal(str(total_10_days)) > 500:
        rate += bonus_rate
    payment = rate * litres
    return payment.quantize(
        Decimal("0.01"), rounding=ROUND_HALF_UP
    )
fat = input("Enter FAT (e.g., 3.25): ")
snf = input("Enter SNF (e.g., 8.50): ")
litres = input("Enter litres supplied today: ")
total = input("Total litres supplied in 10 days: ")
month = int(input("Enter month number (1-12): "))
amount = calculate_payment(fat, snf, litres, total, month)
print("Total Milk Payment: Rs.", amount)
```

output:

```
Python 3.14.8 (tags/v3.14.8:8e6e75d, Sep 30
Enter "help" below or click "Help" above fo
>
====== RESTART: C:/Users/thara/OneDrive/Att
Enter FAT (e.g., 3.25): 2.05
Enter SNF (e.g., 8.50): 7.50
Enter litres supplied today: 3
Total litres supplied in 10 days: 5
Enter month number (1-12): 3
Total Milk Payment: Rs. 98.77
>
```